

Generative AI and Neural Scaling Laws

From Architectural Principles to Engineering Predictability

Introduction to Artificial Intelligence

Universidad EAFIT

September 30, 2025

Seminar Agenda

Introduction

Module 1: The Architecture of Modern Generative AI

Module 2: The Physics of AI: Neural Scaling Laws

Module 3: Scaling Laws in Practice: A Hands-On Laboratory

Conclusion and Future Perspectives

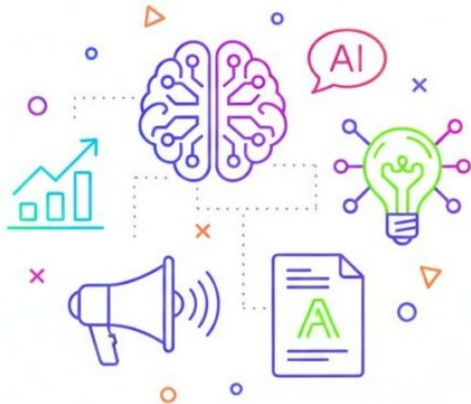
Introduction:

The New AI Paradigm

Welcome and Seminar Objective

- ▶ This seminar aims to deconstruct the current Generative AI revolution from a **first-principles perspective**.
- ▶ We will move beyond the hype to explore the core technologies and the **empirical science** that enables their exponential growth.
- ▶ Our focus is on the interplay between a scalable architecture (the Transformer) and a predictive engineering framework (Neural Scaling Laws).
- ▶ Key takeaways will include design principles, resource management trade-offs, and the optimization of large-scale AI systems.

Generative AI



The Symbiotic Relationship Driving the Revolution

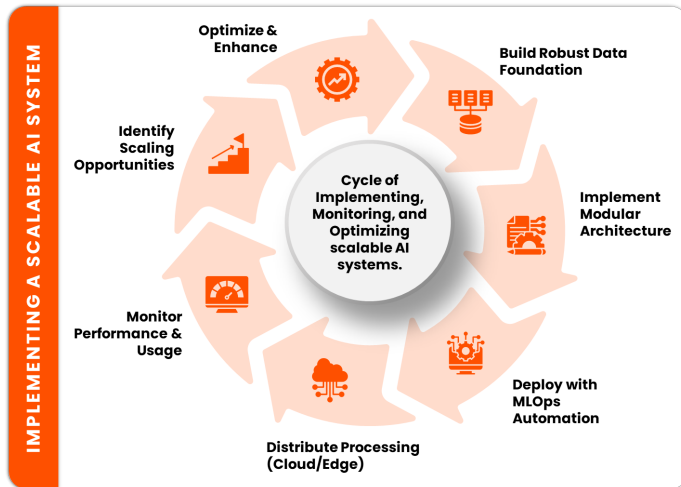
A Scalable Architecture

- ▶ The **Transformer**, introduced in 2017.
- ▶ Its core innovation, the self-attention mechanism, allows for massive parallelization.
- ▶ This architectural choice is the blueprint that enables models to be built at an unprecedented scale.

Predictable Scaling Laws

- ▶ Empirical, power-law relationships discovered through large-scale experiments.
- ▶ They describe how model performance predictably improves with more data, parameters, and compute.
- ▶ This provides a scientific and engineering roadmap for efficiently investing resources.

Scalable Architecture



The Symbiotic Relationship Driving the Revolution

Central Thesis

The current AI explosion is not magic; it's the predictable outcome of combining a highly parallelizable architecture with a scientific understanding of how to scale it effectively.

Module 1:

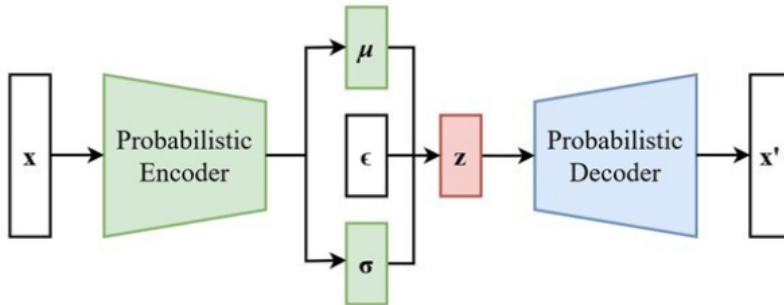
The Architecture of Modern Generative AI

A Formal Taxonomy of Generative Models

- ▶ **Variational Autoencoders (VAEs):** Learn a probabilistic mapping from a high-dimensional data space to a lower-dimensional latent space. They generate data by sampling from this latent space.
 - ▶ **Objective:** Maximize the Evidence Lower Bound (ELBO).

$$\log p(x) \geq \mathbb{E}_{q(z|x)}[\log p(x|z)] - D_{KL}(q(z|x)||p(z))$$

GAN

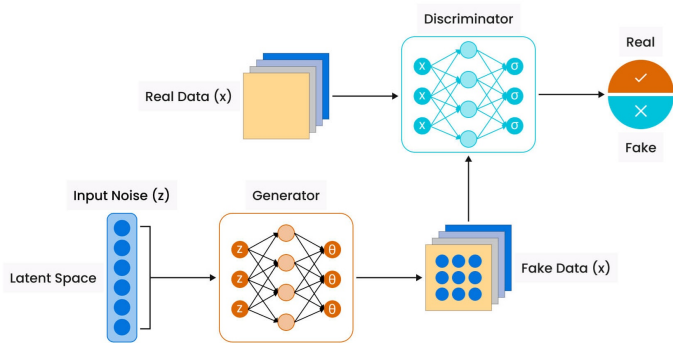


- ▶ **Generative Adversarial Networks (GANs):** A zero-sum game between a Generator (G) that creates data and a Discriminator (D) that tries to distinguish real from fake data.
 - ▶ **Objective:** Minimax value function.

$$\min_G \max_D V(D, G) = \mathbb{E}_{x \sim p_{data}(x)} [\log D(x)] + \mathbb{E}_{z \sim p_z(z)} [\log(1 - D(G(z)))]$$

Variational Autoencoder

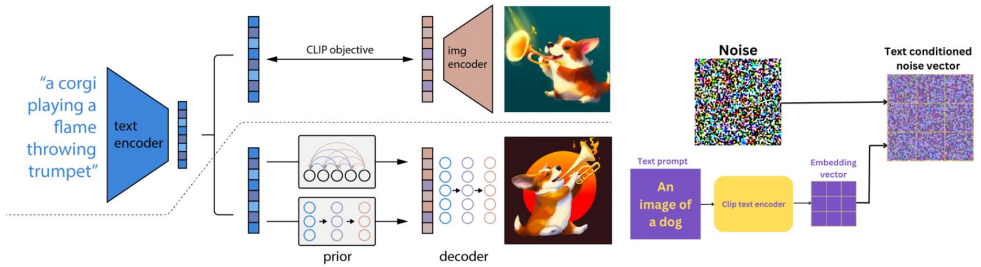
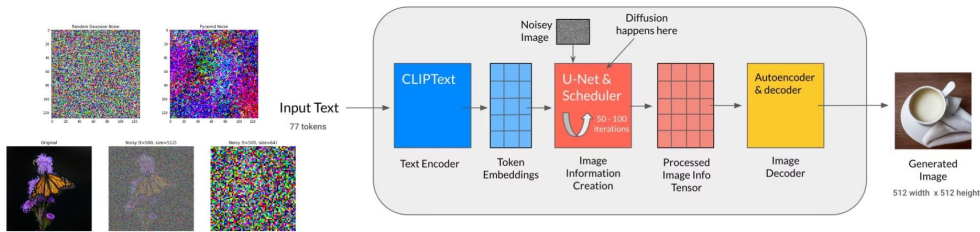
Generative Adversarial Network (GAN)



A Formal Taxonomy of Generative Models

- ▶ **Diffusion Models:** Generate high-quality data by learning to reverse a gradual noising process. They start with pure noise and iteratively denoise it to produce a sample.
 - ▶ **Process:** Forward (noising) process $q(x_t|x_{t-1})$ and a learned reverse (denoising) process $p_\theta(x_{t-1}|x_t)$.

Difussion Model



- ▶ **Autoregressive Models (e.g., Transformers):** Generate data sequentially, where each new element is conditioned on the previous ones.
 - ▶ **Objective:** Maximize the likelihood of the next token given the context.

$$p(x) = \prod_{i=1}^n p(x_i | x_1, \dots, x_{i-1})$$

The Transformer: A Paradigm Shift from Sequential Processing

Before Transformers: RNNs/LSTMs

- ▶ Processed sequences token-by-token.
- ▶ Inherently sequential, making parallelization difficult.
- ▶ Suffered from long-range dependency issues (vanishing/exploding gradients).
- ▶ The path length between tokens grew with sequence length.

The Transformer Revolution (2017)

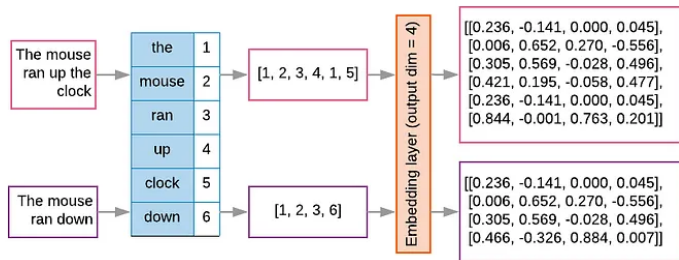
- ▶ Eliminated recurrence entirely.
- ▶ Processed all tokens in parallel using the **self-attention** mechanism.
- ▶ The path length between any two tokens is constant ($O(1)$), allowing for perfect long-range memory.
- ▶ This design was built for modern hardware (GPUs/TPUs).

Input Representation: From Words to Vectors

1. Tokenization and Embedding:

- ▶ Text is segmented into tokens (words, subwords).
- ▶ Each token is mapped to a high-dimensional vector (embedding) via a learnable lookup table. $x_i \in \mathbb{R}^{d_{model}}$.

Tokenization



2. Positional Encoding:

- ▶ Since self-attention is order-agnostic, we must inject positional information.
- ▶ This is done by adding a "position vector" to each token embedding.
- ▶ The original paper used fixed sinusoidal functions:

$$PE_{(pos,2i)} = \sin\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

$$PE_{(pos,2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

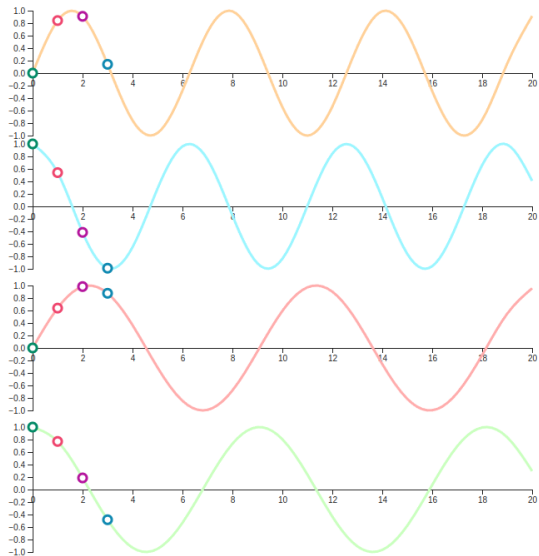
- ▶ **Why sinusoids?** They allow the model to easily learn relative positions, as the encoding of $pos + k$ can be represented as a linear function of the encoding of pos .

Encoding

Sequence	Index of token, k	Positional Encoding Matrix with $d=4$, $n=100$			
		$i=0$	$i=0$	$i=1$	$i=1$
I	0	$P_{00}=\sin(0)$ = 0	$P_{01}=\cos(0)$ = 1	$P_{02}=\sin(0)$ = 0	$P_{03}=\cos(0)$ = 1
am	1	$P_{10}=\sin(1/1)$ = 0.84	$P_{11}=\cos(1/1)$ = 0.54	$P_{12}=\sin(1/10)$ = 0.10	$P_{13}=\cos(1/10)$ = 1.0
a	2	$P_{20}=\sin(2/1)$ = 0.91	$P_{21}=\cos(2/1)$ = -0.42	$P_{22}=\sin(2/10)$ = 0.20	$P_{23}=\cos(2/10)$ = 0.98
Robot	3	$P_{30}=\sin(3/1)$ = 0.14	$P_{31}=\cos(3/1)$ = -0.99	$P_{32}=\sin(3/10)$ = 0.30	$P_{33}=\cos(3/10)$ = 0.96

Positional Encoding Matrix for the sequence 'I am a robot'

Encoding



p0	p1	p2	p3	
0.000	0.841	0.909	0.141	i=0
1.000	0.540	-0.416	-0.990	i=1
0.000	0.638	0.983	0.875	i=2
1.000	0.770	0.186	-0.484	i=3

Positional Encoding

$$PE_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

$$PE_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

Settings: d = 50

The value of each positional encoding depends on the *position* (*pos*) and *dimension* (*d*). We calculate result for every *index* (*i*) to get the whole vector.

The Core Engine: Self-Attention Explained

The goal of self-attention is to produce a new, context-rich representation for each token by making it a weighted sum of all other tokens in the sequence.

Step 1: Create Query, Key, and Value Projections For each input token embedding x_i , we create three vectors by multiplying it with learned weight matrices W_Q, W_K, W_V :

- ▶ **Query** (q_i): What I am looking for. ($q_i = x_i W_Q$)
- ▶ **Key** (k_i): What I contain. ($k_i = x_i W_K$)
- ▶ **Value** (v_i): What I will provide if you attend to me. ($v_i = x_i W_V$)

Step 2: Calculate Attention Scores and Weights

1. **Score:** For a given query q_i , calculate its similarity with every key k_j in the sequence using a dot product: $\text{score}_{ij} = q_i \cdot k_j$.
2. **Scale:** Divide scores by $\sqrt{d_k}$ to stabilize gradients.
3. **Weights:** Apply a softmax function to the scaled scores to get attention weights α_{ij} that sum to 1.

Scaled Dot-Product Attention: The Formula

The entire process can be expressed in a single, highly-optimized matrix operation. Given matrices of queries Q , keys K , and values V :

$$\text{Attention}(Q, K, V) = \underbrace{\text{softmax}}_{\text{Weights}} \left(\underbrace{\frac{QK^T}{\sqrt{d_k}}}_{\substack{\text{Scores} \\ \text{Scale}}} \right) \underbrace{V}_{\text{Values}}$$

- ▶ Q, K, V are matrices of shape (sequence_length, d_k).
- ▶ The QK^T operation computes all dot-product scores in parallel.
- ▶ The output is a new matrix where each row vector is a contextually-aware representation of the corresponding input token.

Multi-Head Attention: Attending to Different Subspaces

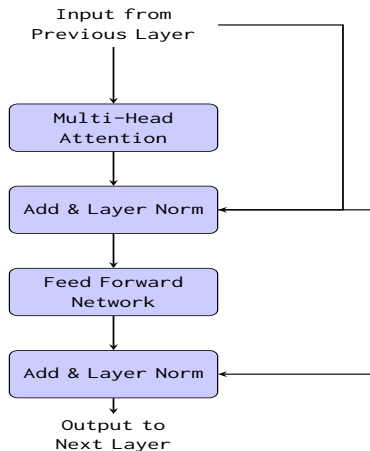
- ▶ It's limiting to have only one way of calculating attention. What if a token needs to attend to syntactic parents, semantic relatives, and co-reference mentions simultaneously?
- ▶ **Solution: Multi-Head Attention.**
- ▶ We run the self-attention mechanism h times in parallel, each with its own set of learned weight matrices (W_{Qi}, W_{Ki}, W_{Vi}) .
- ▶ Each "head" can focus on a different type of relationship in the data.
- ▶ The outputs of all heads are concatenated and linearly projected to produce the final output.

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W_O$$

$$\text{where } \text{head}_i = \text{Attention}(QW_{Qi}, KW_{Ki}, VW_{Vi})$$

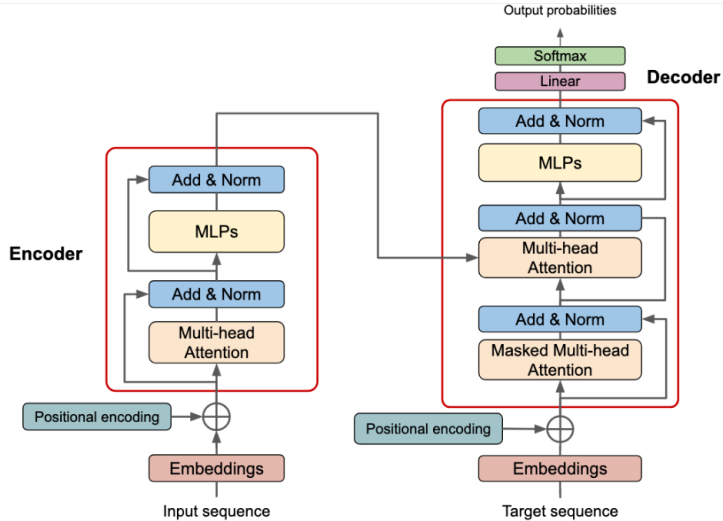
The Full Transformer Block

A complete Transformer layer (or block) combines Multi-Head Attention with other crucial components:



- ▶ **Feed Forward Network (FFN):** A simple 2-layer MLP applied independently to each token representation. It adds modeling capacity.
- ▶ **Add & Norm (Residual Connections and Layer Normalization):** Essential for training deep networks. The "Add" (residual connection) prevents vanishing gradients, while "Norm" stabilizes the activations.

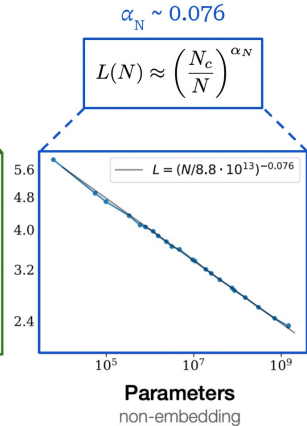
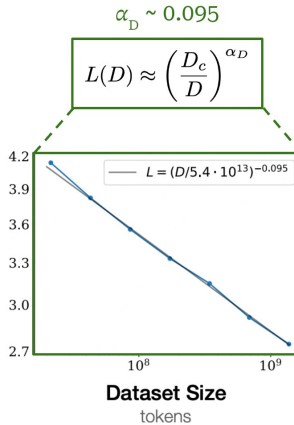
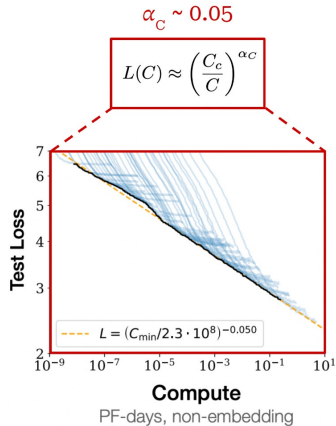
Transformer Blocks



Module 2:

The Physics of AI: Neural Scaling Laws

Scaling Laws



What Are Neural Scaling Laws?

- ▶ **Definition:** Empirical relationships that describe how a model's performance (measured by its cross-entropy loss L) improves predictably as we increase key resources.
- ▶ These relationships typically follow a smooth, predictable **power law**.
- ▶ The three primary resources are:
 1. **Model Size (N):** The number of non-embedding parameters.
 2. **Dataset Size (D):** The number of tokens in the training corpus.
 3. **Compute (C):** The total floating-point operations (FLOPs) used for training.
- ▶ The existence of these laws transforms AI model training from an art into an engineering discipline.

The Power Law Relationship

The core mathematical form of a scaling law is a power law with an irreducible loss term L_0 :

$$L(X) = L_0 + A \cdot X^{-\alpha}$$

- ▶ L is the model's loss (lower is better).
- ▶ X is the resource being scaled (N, D, or C).
- ▶ $\alpha > 0$ is the **scaling exponent**. It determines how rapidly performance improves.
- ▶ L_0 is the irreducible loss, representing the inherent entropy or noise in the data itself (the Bayes error rate).

On a **log-log plot**, this power law becomes a straight line, which makes it easy to verify empirically:

$$\log(L(X) - L_0) = \log(A) - \alpha \log(X)$$

This is the equation of a line ($y = c + mx$) with a negative slope of $-\alpha$.

The First Major Study: Kaplan et al. (OpenAI, 2020)

The paper "Scaling Laws for Neural Language Models" was a landmark study that systematically analyzed these trends for Transformers.

Key Findings:

- 1- Model performance scales smoothly and predictably with N , D , and C , each following a power law when other factors are not bottlenecks.
 - ▶ $L(N) \propto N^{-\alpha_N}$ with $\alpha_N \approx 0.076$
 - ▶ $L(D) \propto D^{-\alpha_D}$ with $\alpha_D \approx 0.095$

The First Major Study: Kaplan et al. (OpenAI, 2020)

The paper "Scaling Laws for Neural Language Models" was a landmark study that systematically analyzed these trends for Transformers.

Key Findings:

- 2- **Larger models are more sample-efficient.** A bigger model requires less data to reach the same level of performance as a smaller model.
- 3- The primary bottleneck appeared to be model size (N).

The First Major Study: Kaplan et al. (OpenAI, 2020)

The Implication ("Bigger is Better")

For optimal performance, one should prioritize making the model as large as possible and train it for as long as compute allows. This philosophy guided models like GPT-3.

The Paradigm Shift: Hoffmann et al. (DeepMind, 2022)

The "Chinchilla" paper addressed a more practical question: "Given a fixed compute budget (FLOPs), what is the optimal allocation between model size (N) and dataset size (D)?"

Methodology:

1. Define the training compute budget: $C \approx 6ND$.
2. Model the loss as a function of both N and D:

$$L(N, D) = L_0 + \frac{A}{N^\alpha} + \frac{B}{D^\beta}$$

3. Use the compute constraint ($D \propto C/N$) to find the values of N and D that minimize L for a fixed C .

The Paradigm Shift: Hoffmann et al. (DeepMind, 2022)

The Stunning Finding: The optimal strategy is to scale N and D in **approximately equal proportion**. For every doubling of model size, the number of training tokens should also be doubled.

The Implication ("Balance is Better")

Previous large models like GPT-3 (175B params, 300B tokens) were severely undertrained. A much smaller model trained on more data could outperform them.

Chinchilla vs. The Old Paradigm

Chinchilla (70B params, 1.4T tokens) was trained with the same compute budget as Gopher (280B params, 300B tokens) but outperformed it on virtually all benchmarks.

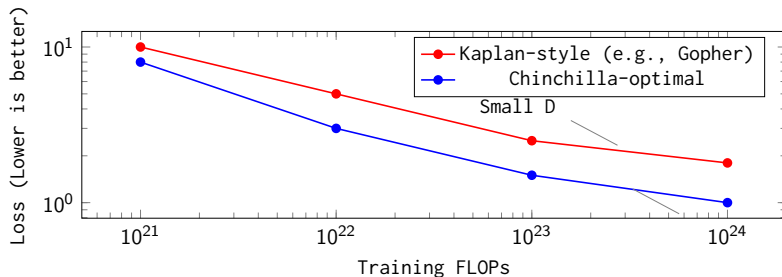


Figure: Conceptual illustration of compute-optimal training. For the same FLOPs, a balanced model (Chinchilla) achieves lower loss.

Module 3:

Scaling Laws in Practice: A Hands-On Laboratory

Workshop Goal: Empirically Verify a Scaling Law

We will conduct a small-scale scientific experiment to test a hypothesis derived directly from the scaling laws papers.

- ▶ **Hypothesis:** The validation loss of a Transformer model on a text classification task will decrease as a power law of its number of parameters ($L \propto N^{-\alpha}$), assuming the training dataset and hyperparameters are held constant.

Workshop Goal: Empirically Verify a Scaling Law

► Methodology:

1. Select a standard dataset (e.g., IMDB reviews).
2. Define a series of Transformer architectures with varying numbers of parameters (N).
3. Train each model for a fixed number of steps.
4. Record the final validation loss and the number of trainable parameters for each model.

► Analysis: Plot the results on a log-log scale and perform a linear regression to find the slope, which is our empirical estimate of the scaling exponent $-\alpha$.

The Modern AI Professional's Toolkit

- ▶ **Google Colab:** A cloud-based notebook environment providing free access to GPUs, essential for training neural networks.
- ▶ **Hugging Face Ecosystem:** The industry standard for working with Transformer models.
 - ▶ 'datasets': For efficiently loading and preprocessing massive datasets.
 - ▶ 'transformers': Provides high-level APIs for models ('AutoModel'), configurations ('AutoConfig'), and a powerful, easy-to-use training loop ('Trainer').
 - ▶ 'evaluate': For standardized model evaluation metrics.

Step 1: Environment Setup and Data Loading

```
1 # Install necessary libraries
2 !pip install transformers datasets evaluate accelerate -q
3
4 import torch
5 from datasets import load_dataset
6 from transformers import AutoTokenizer, AutoConfig,
   AutoModelForSequenceClassification, Trainer, TrainingArguments
7
8 # Load and prepare a subset of the IMDB dataset
9 dataset = load_dataset("imdb", split="train[:5000]")
10 ...
11
```

Step 2: The Core Experiment Loop

```
1 import pandas as pd
2
3 # Define different model configurations to vary size (N)
4 model_configs = {
5     'tiny': {'num_hidden_layers': 2, 'hidden_size': 128, '
6             num_attention_heads': 2},
7     'small': {'num_hidden_layers': 4, 'hidden_size': 256, '
8              num_attention_heads': 4},
9     'base': {'num_hidden_layers': 6, 'hidden_size': 512, '
10             num_attention_heads': 8},
11     'large': {'num_hidden_layers': 8, 'hidden_size': 768, '
12              num_attention_heads': 12},
13 }
```

Step 3: Visualization and Analysis

```
1 import matplotlib.pyplot as plt
2 import numpy as np
3 from scipy.stats import linregress
4
5 # Extract data for plotting
6 params = results_df['params'].values
7 losses = results_df['loss'].values
8
9 # Convert to log scale
10 log_params = np.log10(params)
11 log_losses = np.log10(losses)
12
13 # Perform linear regression on the log-log data
14 ...
15
```

Discussion of Workshop Results

- ▶ **Expected Outcome:** The data points on the log-log plot should align closely to the fitted straight line, visually confirming the power-law relationship. The R-squared value should be high (e.g., > 0.95).
- ▶ **Interpreting α :** The calculated slope ($-\alpha$) quantifies how efficiently performance scales with model size in this specific experimental setup. A larger α means performance improves more rapidly with size.

Discussion of Workshop Results

► Sources of Noise and Limitations:

- **Stochasticity:** The inherent randomness of weight initialization and data shuffling introduces noise.
- **Small Scale:** Our experiment uses a tiny fraction of the data and compute of large-scale studies. The "true" scaling law only becomes clear over many orders of magnitude.
- **Hyperparameters:** We kept hyperparameters (learning rate, epochs) fixed. At different scales, these might need to be tuned for optimal results.

- **Critical Thinking Question:** How would you design a follow-up experiment to investigate the scaling law for dataset size ($L \propto D^{-\beta}$)?

Conclusions:

Conclusion and Future Perspectives

Recapitulation of the Central Thesis

- ▶ The current AI revolution is not an accident. It is the direct result of the convergence of a **massively parallelizable architecture** (the Transformer) and **predictable engineering principles** (neural scaling laws).
- ▶ The Transformer architecture provided the **blueprint** for building models at immense scale.
- ▶ Scaling laws provided the **instruction manual** for how to invest billions of dollars in compute and data resources efficiently and predictably.

The Engineer's Role in a World of Foundational Models

The engineering focus has fundamentally shifted:

- ▶ From: Algorithm and architecture design from scratch.
To: Adapting, fine-tuning, and optimizing the deployment of massive pre-trained models.
- ▶ From: Logic-based programming.
To: Managing emergent behaviors and prompting complex systems.
- ▶ From: Model performance optimization.
To: Compute, data, and financial resource optimization guided by scaling laws.

Ethical and Security Frontiers

As these models scale, so do the challenges. We must be vigilant about:

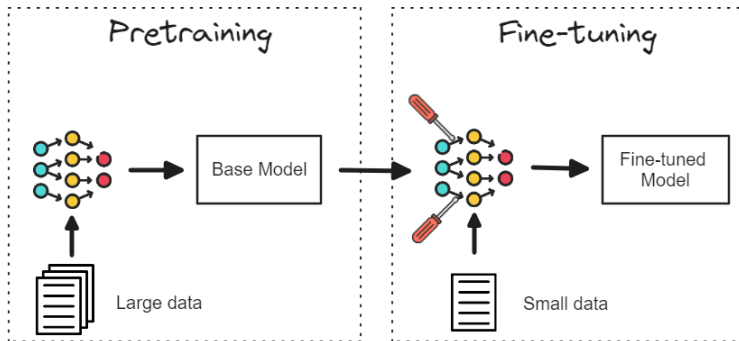
- ▶ **Algorithmic Bias:** Models are mirrors of their training data. Biases present in the data (societal, historical, etc.) will be learned and can be amplified, leading to inequitable outcomes.
- ▶ **Opacity and Explainability:** The "black box" nature of models with billions of parameters is a fundamental barrier to trust, accountability, and debugging. How can we trust a decision we cannot understand?
- ▶ **Potential for Misuse:** The capacity to generate highly convincing text, images, and code at scale creates powerful tools for misinformation, social engineering, and cyberattacks.

Future Perspectives

- ▶ **Breaking the Scaling Laws?:** Will performance continue to scale predictably, or are we approaching a plateau due to data quality/quantity limits or architectural bottlenecks?
- ▶ **Multimodality:** The next frontier is models that seamlessly process and generate information across text, images, audio, and video (e.g., Gemini, GPT-4V).
- ▶ **Agentic AI:** Moving from passive prediction to active agents that can use tools, browse the web, and execute complex, multi-step tasks in the real world.
- ▶ **Efficiency:** A major research thrust is to achieve the same capabilities with drastically less compute and data, through better architectures, training algorithms, and quantization.

La Era del Pre-entrenamiento y Fine-Tuning

Large Language Model



Thank You

Questions?



Escuela de
Ciencias Aplicadas
e Ingeniería